

THEOREMFORCES

再現可能な Lean 証明のための 公開 certificate layer

A Public Certificate Layer for Reproducible Lean Proofs

Status Public beta design paper

Version 0.1.0

Date 2026-08-07

本書は、TheoremForces の目的、trust model、certificate 仕様、judge architecture、semantic provenance、security、governance、および 2026-2027 年の実行計画を示す。実装済み機能と目標状態を明確に区別し、Lean acceptance を数学的 endorsement と混同しないことを設計の中心に置く。

TheoremForces Project

<https://theoremforces.org/>

Independent project. Not affiliated with, endorsed by, or sponsored by Lean FRO, the Lean project, or the mathlib maintainers.

要旨

TheoremForces は、人間と AI が作る Lean 4 の conjecture と proof body を、固定された Lean / Mathlib 環境で検証し、その結果を再現・引用・再利用可能な certificate として記録するための独立 platform である。中心的な問いは「誰が一番速く解いたか」だけではない。何が、どの環境で、どの axiom と依存関係のもとに検証され、第三者がどこまで再現できるかである。

本書は certificate を単一の badge として扱わず、(1) kernel acceptance、(2) artifact provenance、(3) active environment での reverification、(4) informal intent review、(5) curation の五層に分ける。公開 beta の現状には、外部利用、review quorum、artifact reconstruction、sandbox enforcement、toolchain migration に未解決課題がある。したがって今後の優先順位は、機能拡張や課金ではなく、trust reset、research lighthouse、external adoption、review network、条件付き monetization の順とする。

Abstract

TheoremForces is an independent platform for checking Lean 4 conjectures and proof bodies in a pinned Lean / Mathlib environment and preserving the results as reproducible, citable, and reusable certificates. The central question is not only who solved a problem, but exactly what was checked, under which environment and axiom policy, with which certificate dependencies, and to what extent an independent party can reproduce the result.

This white paper separates five assurance layers: kernel acceptance, artifact provenance, reverification under the active environment, review of alignment with informal intent, and mathematical curation. It documents both the public-beta system and a proposed target architecture. The execution strategy is deliberately gated: repair trust-critical gaps first, demonstrate value through real research collections second, grow a small retained external community third, and open paid pilots only after reliability and operational thresholds are met.

文書の読み方

- **Current:** 2026-08-07 時点で公開 UI、公開 API、公開 repository から確認できる状態。
- **Target:** 本書が提案する certificate schema v2 と production architecture。
- **Gate:** 次の phase へ進むための measurable exit condition。
- 本書は private web application と isolated judge source code の完全な security audit ではない。

目次

1	問題設定	1
1.1	証明が生成される時代の provenance gap	1

1.2	対象ユーザー	1
1.3	非目標	2
2	設計原則	2
3	Assurance model	3
3.1	五つの独立した assurance layer	3
3.2	Acceptance predicate	3
3.3	Axiom policy	4
4	System architecture	4
4.1	Current public description	4
4.2	Target pipeline	4
5	Certificate schema v2	6
5.1	Design goals	6
5.2	Canonicalization	7
5.3	Lifecycle and supersession	7
5.4	Independent verifier	8
6	Semantic provenance and review	8
6.1	なぜ kernel check と意味 review を分けるのか	8
6.2	Review dimensions	9
6.3	AI disclosure	9
7	Security and abuse resistance	9
7.1	Threat model	9
7.2	Fail-closed resource enforcement	10
7.3	Secret handling and incident response	10
8	Public beta snapshot and trust reset	11
8.1	Snapshot: 2026-08-07	11
8.2	Observed trust gaps	11
8.3	Trust reset plan	11
9	Research workflow and interoperability	12
9.1	Flagship collection	12

9.2	Relationship to mathlib	12
9.3	Academic citation	13
9.4	Preservation tiers	13
10	Governance and sustainable operation	13
10.1	Small-team operating model	13
10.2	Change governance	14
10.3	Business boundary	14
11	Roadmap: 2026-08 to 2027-07	14
11.1	Phase 0: first 48 hours	15
11.2	Phase 1: trustworthy beta	15
11.3	Phase 2: research lighthouse	16
11.4	Phase 3: review and retention	16
11.5	Phase 4: paid pilot decision	16
11.6	Phase 5: evidence-driven scale	16
12	Metrics and decision gates	16
12.1	North Star	16
12.2	Dashboard	17
12.3	Feature decision rule	17
13	Open questions	17
14	Conclusion	18
付録 A	Proposed certificate v2 example	18
付録 B	Verification algorithm	19
付録 C	Operational SLO proposal	19
付録 D	Glossary	20
References		21

1 問題設定

1.1 証明が生成される時代の provenance gap

Lean は dependent type theory に基づく interactive theorem prover であり、elaborator や tactic が生成した proof term は小さな kernel によって検査される [13, 14]。この構造は、tactic や AI が複雑化しても、最終的な formal derivability を kernel で確認できるという強い基盤を持つ。

しかし、kernel が受理したという事実だけでは、研究上必要な問いのすべてには答えない。

- formal statement は、研究者が意図した informal theorem と一致しているか。
- proof はどの Lean / Mathlib revision と import set で検査されたか。
- sorryAx、custom axiom、compiler trust、既存 certificate への依存は何か。
- judge に投入された exact source と、後から UI が再構成する source は同一か。
- toolchain 更新後も通るか。通らない場合、historical validity と current compatibility をどう分けるか。
- AI が生成した artifact を、人間の review や数学的価値判断とどう区別するか。

TheoremForces は、この gap を certificate ledger と semantic provenance によって埋める。目標は「AI が真理を宣言する platform」ではなく、「Lean が何を受理し、その artifact がどのように作られ、何を意味すると人間が判断したかを分離して残す platform」である [1, 7]。

1.2 対象ユーザー

利用者	主要 job	必要な assurance
数学者	informal problem を Lean statement と proof project に分解する	intent alignment、citation、共同作業
Lean formalizer	proof body を検査し、再利用可能な theorem chain を作る	exact environment、imports、axioms
AI theorem prover 開発者	generated proof を外部 judge で評価し、benchmark record を残す	API、rate limit、machine-readable certificate
研究 group / lab	private work を保存し、公開時に選択的に certificate 化する	access control、export、migration
第三者 verifier	platform operator を全面的に信頼せず検算する	content address、manifest、one-command verifier
reviewer / curator	formal statement と数学的意図・価値を評価する	review trail、conflict policy、retraction

1.3 非目標

TheoremForces は mathlib の代替ではない。一般的で再利用可能な数学は、適切な review を経て mathlib に upstream されるべきである。TheoremForces は、その前後で発生する conjecture、attempt、pinned build artifact、AI disclosure、citation metadata を記録する staging / certificate layer である [2, 18]。

また、Accepted は novelty、importance、peer review、publication readiness を意味しない。Seeds は money、token、security、redeemable asset ではなく、platform 内の reputation signal に限定する。marketplace、cash bounty、off-platform payment は scope 外である [2, 8]。

2 設計原則

P1. Lean accepted は必要条件であり、数学的 endorsement ではない

Kernel acceptance、artifact reproduction、intent review、curation を別々の state として表示する。一つの緑 badge に統合しない。

P2. Exact bytes over regenerated appearance

証明書が hash する対象は、後日の template で再生成した表示ではなく、judge が実際に受け取った byte-for-byte source と bundle である。表示も保存 artifact を source of truth とする。

P3. Pinned is not latest

Active pinned environment と upstream stable release は別の概念である。historical certificate は元の epoch で再現し、current compatibility は migration campaign の結果として別記する。

P4. Fail closed at trust boundaries

Hash、dependency、axiom parse、resource limit、webhook identity のどれかを確認できない場合、Accepted や fully reproducible と表示しない。未知を成功へ丸めない。

P5. Public verification must not require operator trust

Certificate JSON、artifact、manifest、Merkle inclusion proof、verifier を公開し、第三者が独立に検算できる経路を用意する。SLSA provenance と in-toto attestation の考え方を参照し、何が、いつ、どの builder と inputs から生成されたかを machine-readable に残す [20, 21]。

P6. Research value before feature surface

North Star は conjecture 総数や AI-generated submission 数ではない。外部利用者によって作られ、別の proof または研究成果から再利用・引用された provenance-complete certificate 数である。

3 Assurance model

3.1 五つの独立した assurance layer



図 1: 一つの badge に畳み込まない assurance stack

L1 は Lean の kernel による derivability の確認である。L2 は、その確認対象と実行環境の provenance を第三者が検算できることを表す。L3 は時間軸を導入し、historical acceptance と current compatibility を分離する。L4 は人間が意図との一致を review した状態、L5 は数学的価値を選定した状態である。

この分離によって、たとえば次の表現が可能になる。

- Lean accepted / artifact mismatch / intent unreviewed
- Lean accepted / provenance complete / needs migration
- Lean accepted / provenance complete / intent aligned / curated
- Lean accepted / provenance complete / intent misaligned / useful accidental lemma

3.2 Acceptance predicate

Formal statement を s 、proof body を p 、direct certificate imports を I 、generator version を v 、environment epoch を E 、axiom whitelist を W とする。生成 source と judge bundle を

$$A = G_v(s, p, I, E), \quad (1)$$

$$B = \text{Bundle}(A, I, E) \quad (2)$$

と定義する。 $J_E(B)$ を isolated judge の結果、 $X_E(A)$ を `#print axioms` から得た transitive axiom set、 $D(B)$ を bundle から計算した direct dependency set、 H を SHA-256 とする。新規 submission の Accepted 条件は、概念的には次である。

$$\begin{aligned}
\text{Accept}(s, p, I, E) \iff & J_E(B) = \text{success} \\
& \wedge X_E(A) \subseteq W \\
& \wedge \text{sorryAx} \notin X_E(A) \\
& \wedge D(B) = I \\
& \wedge H(A) = h_A \wedge H(B) = h_B \\
& \wedge \text{LimitsEnforced}(J_E).
\end{aligned} \tag{3}$$

重要なのは、単なる process exit code 0 ではない点である。実行対象、axiom report、dependency、resource policy、recorded hash が同一 transaction / attestation chain に結び付いて初めて L1 と L2 が成立する。

3.3 Axiom policy

Lean は definition が直接・間接に依存する axiom を `#print axioms` で列挙する。公式 reference は `sorryAx`、`compiler trust`、`custom axiom` と、標準的な `propext`、`Classical.choice`、`Quot.sound` を区別している [15, 16]。

TheoremForces の policy は次を満たすべきである。

1. `sorryAx` は whitelist に関係なく hard reject。
2. parse failure / missing report は success ではなく infrastructure failure。
3. standard axiom、custom axiom、imported certificate constant を UI 上で別 category にする。
4. certificate dependencies は axiom list の文字列推測ではなく、server-resolved bundle manifest から確定する。
5. parser version と raw output hash を certificate に含める。

4 System architecture

4.1 Current public description

公開 documentation によれば、web application は Next.js on Cloudflare Workers、database は Cloudflare D1、Lean build は外部 isolated judge が担当する。web server 自身は Lean を実行せず、fixed namespace、static filter、type-checking trailer、pinned toolchain、HMAC-authenticated callback を組み合わせる [2, 3, 4]。

公開 judge repository は Lean v4.13.0 と Mathlib v4.13.0 を pin し、GitHub Actions workflow で targeted lake build、axiom whitelist enforcement、machine-readable axiom report を実装している [10]。一方、公開 submission は isolated arm64 worker を示しているため、public repository と production topology の関係を architecture document で明示する必要がある。

4.2 Target pipeline

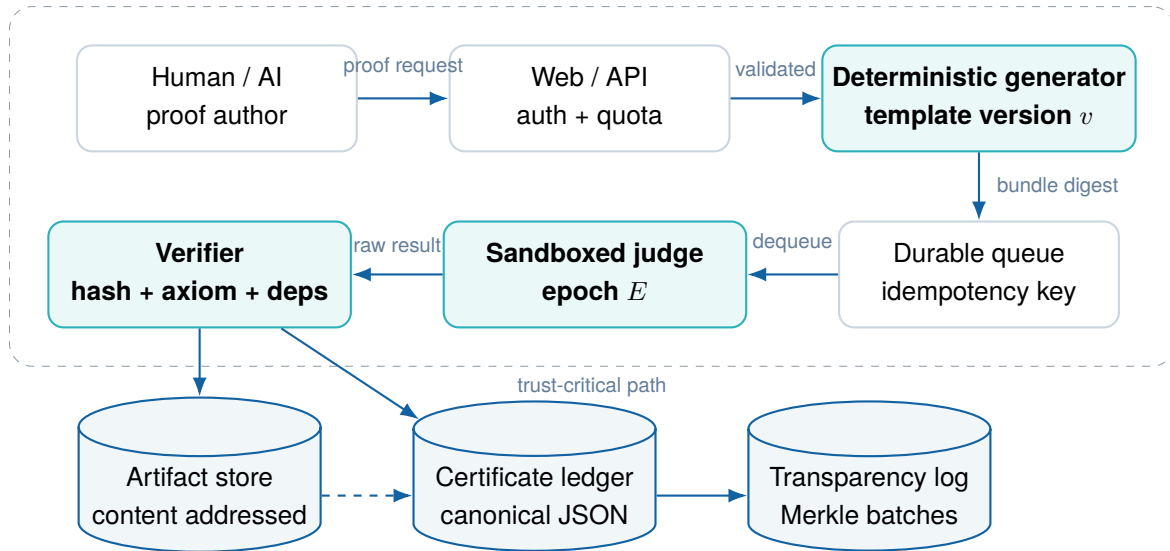


図 2: 提案する end-to-end certificate pipeline

4.2.1 Submission intake

Request は authenticated identity、conjecture id、proof body、AI disclosure、selected certificate imports、client idempotency key を含む。server は quota と content-size cap を先に適用し、static filter の結果を記録する。static filter は defense-in-depth であり、Lean soundness の完全な証明とは扱わない。

4.2.2 Deterministic generation

Generator は statement、proof body、server-resolved imports、namespace、type-checking trailer、axiom command を順序を固定して出力する。生成 source は直ちに hash し、template version とともに immutable artifact store へ保存する。後日の UI は source を再生成せず、この blob を表示する。

Listing 1: 生成 artifact の概念例

```

-- generator: tf-wrapper-v2
-- environment: lean-4.32.2-mathlib-<commit>
-- submission: <id>

import Mathlib
import Theoremforces.Ledger.Cert_<dependency>

namespace Theoremforces.Conjectures.<NS>.Submissions.<ID>
<proof body bytes, unchanged>
def solution_target : <target type> := <answer name>
#print axioms solution_target
end Theoremforces.Conjectures.<NS>.Submissions.<ID>

```

4.2.3 Sandboxed judge

Judge image は digest で pin し、起動時 preflight で cgroup v2、network namespace、read-only base filesystem、non-root uid、ephemeral workspace、PID / memory / CPU / wall-clock / output

cap を確認する。一つでも enforce できなければ production readiness check を失敗させる。

4.2.4 Verification and minting

Verifier は worker が返した status をそのまま信用しない。job identity、bundle hash、environment digest、axiom report、dependency manifest、resource policy result を照合する。minting と public status update は idempotent transaction とし、callback replay で二重 certificate や二重 reputation credit が発生しないようにする。

4.2.5 Transparency log

Certificate hash は UTC batch の Merkle tree に追加し、inclusion proof と signed tree head を公開する。Merkle root を同一 database に保存するだけでは operator による履歴改変検出が弱いので、root を独立した Git repository、release artifact、または別 provider の immutable storage に publish する。RFC 9162 の transparency log は certificate domain は異なるが、append-only Merkle log の設計参照になる [22]。

5 Certificate schema v2

5.1 Design goals

v2 schema は「metadata がある」だけでなく、「第三者が同じ object graph を再構成できる」ことを目標とする。SLSA が provenance を artifact の origin、time、builder、inputs に関する verifiable information と定義する考え方を採り入れる [20]。

Object	Required fields	Verification purpose
Statement	canonical text, hash, conjecture id, intent revision	何を証明したかを固定
Proof body	exact bytes, hash, author disclosure	user input を固定
Generated artifact	source blob digest, module path, template id	judge input の再構成を不要にする
Environment	Lean tag + digest, Mathlib commit, manifest hash, epoch	toolchain を同定
Builder	image digest, architecture, sandbox policy, run id	実行主体と制限を同定
Axioms	normalized names, raw report hash, parser version	trust assumptions を監査
Dependencies	direct / transitive cert hash, module hash, constant	graph と taint を追跡
Result	exit status, timestamps, log digest, resource outcome	execution result を検算
Transparency	batch id, leaf index, inclusion proof, signed root	append-only history を検算

5.2 Canonicalization

Certificate payload P は UTF-8、lexicographically sorted object keys、preserved array order、明示的 null policy を持つ canonical JSON とする。certificate hash は自身の hash field を除いた canonical bytes に対して計算する。

$$h_C = \text{SHA256}(\text{CanonicalJSON}(P \setminus \{\text{certificateHash}\})). \quad (4)$$

Array order の意味を schema ごとに規定する。たとえば direct imports は request order ではなく canonical constant で sort してよいが、Merkle audit path は順序を保持しなければならない。canonicalization implementation は golden vectors を公開し、少なくとも二言語で同じ hash を生成することを test する。

5.3 Lifecycle and supersession

Immutable certificate を後から silent edit しない。問題が発見された場合は状態 record を append する。

- **active**: 現在有効な historical acceptance record。

- **superseded**: 新しい environment / corrected provenance certificate が後継。
- **invalidated**: acceptance predicate または artifact integrity に重大な欠陥。
- **retracted**: informal explanation / authorship / policy 上の理由。kernel record は履歴として残す。
- **legacy-incomplete**: 過去 schema に必要 field がなく、完全再現を主張しない。

Status event 自体にも timestamp、reason code、actor、evidence link、previous event hash を持たせる。

5.4 Independent verifier

Reference CLI は次の UX を目標とする。

Listing 2: 目標とする verification UX

```
theoremforces verify tcl-cert-20260605-0d2nux

[ok] certificate canonical hash
[ok] statement and proof-body hashes
[ok] generated Lean source hash
[ok] judge bundle manifest
[ok] 4 direct certificate dependencies
[ok] Merkle inclusion proof
[ok] sandbox policy attestation
[ok] lake build in recorded environment
[ok] normalized axiom report matches certificate
```

Verification mode は三段階にする。

1. **metadata**: download と hash / Merkle / dependency 検算だけ。
2. **rebuild**: recorded container / toolchain で lake build を再実行。
3. **migration**: selected target epoch で rebuild し、新しい compatibility report を作る。

6 Semantic provenance and review

6.1 なぜ kernel check と意味 review を分けるのか

Lean kernel は formal statement の derivability を確認する。formal statement が informal theorem の翻訳として妥当か、仮定が強すぎないか、定義が意図した対象を表すか、数学的価値があるかは別問題である。TheoremForces の公開 model は Wild、Registry、Curated の三層を提案している [7]。

Wild

AI-generated conjecture、failed attempt、scaffold、unreviewed certificate を探索として許容する。

Registry

Lean acceptance と machine-verifiable provenance を持つ record。数学的 endorsement ではない。

Curated

intent、axiom safety、essential dependencies、explanation、research / educational value を人間が確認する。

6.2 Review dimensions

Review を一票の like にしない。各 reviewer は少なくとも次を独立に評価する。

1. **Intent alignment:** informal theorem と formal target が一致するか。
2. **Assumption adequacy:** 不要に強い仮定、vacuous premise、empty type 依存がないか。
3. **Definition fidelity:** custom definition が標準的数学概念を正しく表すか。
4. **Dependency safety:** imported certificate の status と semantic role が適切か。
5. **Explanation quality:** 数学者が proof の役割と限界を理解できるか。

Canonical review status は reviewer eligibility、self-review exclusion、conflict of interest、blocking dispute、retraction を考慮する。公開 documentation の 5 人 quorum は high assurance には妥当だが、初期 community では quorum が成立しにくい。そのため UI は 0/5 を単なる未達として見せるだけでなく、各 dimension の partial progress と reviewer shortage を示す。

6.3 AI disclosure

AI 使用は許可し、acceptance を disclosure で差別化しない。ただし、human authorship、AI drafting、repair、autonomous submission の level を machine-readable に記録する。reviewer は「AI を使ったか」ではなく、artifact、intent、dependency、explanation を評価する。

重要なのは、AI disclosure を proof validity の代用品にも、不正検出 detector にも使わないことである。自己申告は provenance の一部であり、kernel acceptance は実際の Lean result、semantic endorsement は review record によって決まる。

7 Security and abuse resistance

7.1 Threat model

Threat	Impact	Required controls
Fake acceptance	証明なしで certificate / reputation を取得	exact job identity、commit / bundle hash、verifier-side result fetch、idempotent mint
Namespace escape	他 submission、statement、judge package を改変	server-owned imports、wrapper namespace、token filter、minimal build context
Elaboration-time code execution	credential / filesystem / network 侵害	no network、non-root、read-only root、seccomp、ephemeral workspace
Resource exhaustion	queue outage、費用増大、他 user の starvation	cgroup CPU / memory / PID、timeout、size cap、fair queue、quota

Threat	Impact	Required controls
Axiom bypass	sorryAx や custom axiom を clean と表示	semantic axiom report、fail-closed parser、raw report hash、regression corpus
Artifact substitution	表示 source と judge source が不一致	exact source blob、content address、bundle manifest、independent verifier
Dependency omission	imported theorem を base environment と誤表示	server-resolved import manifest、direct/transitive closure check
Callback replay / confusion	二重 credit、別 submission への結果付替え	HMAC、nonce / idempotency、submission + job + digest binding
Account takeover	private ledger / identity 侵害	passkey / MFA、secure cookie、reset hardening、session revocation、audit log
Secret disclosure	infrastructure takeover	managed secret store、short-lived token、rotation、redaction runbook
Database compromise	private data exposure / ledger mutation	least privilege、encrypted backup、PII separation、external transparency root
Supply-chain compromise	judge image / action / dependency 改変	digest pin、signed build provenance、dependency review、two-person release approval

7.2 Fail-closed resource enforcement

Public sample logs observed on 2026-08-07 contain a warning that memory-limit capability was unavailable, while the job completed successfully [11]. This observation does not by itself establish exploitability or root cause, but it is sufficient to define a P0 control: production workers must refuse jobs when declared resource limits are not actually enforced.

Production invariant

LimitsEnforced = false は warning ではなく infrastructure failure とする。Proof failure、time-out、infrastructure failure、policy rejection を別 status にし、user に修正可能性を正確に伝える。

7.3 Secret handling and incident response

Credential、VPN profile、invite link、service token を chat、Issue、commit、shared note に貼らない。secret manager と short-lived credential を使用する。漏えいが疑われる場合は、削除より先に失効、scope 確認、access-log review、再発行、incident timeline、再発防止を行う。operator account には MFA / passkey を必須とする。

8 Public beta snapshot and trust reset

8.1 Snapshot: 2026-08-07

公開 API と repository から確認した current snapshot は次の通りである [9, 10, 17]。

Indicator	Observed value	Interpretation
Accepted certificates	387	supply は既に存在
Distinct external accepted provers	2	external adoption は初期段階
Reviewed certificates	0	semantic review network 未成立
Curated certificates	0	curation 実績未成立
No-disallowed-axiom rate	95.1% (368/387)	policy metric は存在、category audit が必要
Active pinned environment	Lean / Mathlib v4.13.0	historical reproducibility を保持すべき
Upstream stable Lean	v4.32.2	active と latest を分離表示すべき
Open Issues / PRs in public judge repo	0 / 0	public feedback / roadmap 導線が弱い
Public pricing	beta, paid disabled	monetization より reliability gate が先

8.2 Observed trust gaps

公開 sample submission では、UI が再計算した generated file hash と stored hash の不一致が表示される。また、その certificate では environment epoch や lake manifest hash 等の provenance field が未記録の例がある [11, 12]。これは「proof が誤り」と即断する根拠ではないが、「表示された source が exact judge input であり、第三者が完全に再現できる」という L2 claim を弱める。

さらに active pinned environment v4.13.0 に対し、公開 metric label は *Reverified on latest Lean* と表現する。2026-08-07 時点の upstream stable は v4.32.2 であるため、*Reverified on active TheoremForces environment* と *Upstream version lag* に分ける方が正確である [5, 17]。

8.3 Trust reset plan

- 新規機能と paid launch を一時 freeze。
- 全 387 certificate の artifact、dependency、environment、batch provenance を機械監査。
- complete / legacy-incomplete / artifact-mismatch / dependency-mismatch / reverification-failed に分類。
- mismatch / incomplete を fully reproducible と表示しない。

5. exact generated source を immutable blob として保存・配信。
6. schema v2 と verifier CLI を導入。
7. sandbox limit preflight を fail closed に変更。
8. v4.32.2 への shadow migration を実施し、failure taxonomy を公開。
9. stats API を唯一の集計元にして page 間の counter / label を統一。
10. public repository の branding、architecture、security policy、Issue workflow を現行化。

Trust reset exit gate

新規 certificate の exact artifact hash、dependency record、sandbox enforcement が 100% 一致し、restore drill が成功し、public label inconsistency が 0、critical integrity incident なしで 14 日間運用できた時点で完了とする。

9 Research workflow and interoperability

9.1 Flagship collection

TheoremForces の価値は certificate 数ではなく、実研究の problem decomposition、proof reuse、citation に現れる。flagship collection は最低限、次を一つの graph として提示する。

- informal problem と paper / preprint / notebook の source
- formal target と intent mapping
- definition、auxiliary lemma、bridge lemma、target theorem の semantic role
- certificate dependency DAG
- exact reproducibility bundle
- AI assistance と human contribution disclosure
- known limitation、review status、migration status
- BibTeX / LaTeX / Markdown / JSON citation
- mathlib へ upstream すべき general-purpose component

初期候補は、公開 ledger に連鎖が見える Eisenstein integer / quadratic form 系列である。第二候補として、研究者が実際に抱える inverse problem、geometry、analysis 等を選び、単発の toy problem と研究上の target theorem を区別する。

9.2 Relationship to mathlib

Mathlib は Lean 4 の shared mathematical library であり、source、cache、dependency workflow を公開している [18]。TheoremForces は次の bridge を提供する。

1. exploratory certificate から reusable lemma を特定する。
2. dependency と axiom status を整理する。

3. naming、generality、documentation、tests を mathlib contribution standard に合わせる。
4. TheoremForces certificate を approval として扱わず、通常の mathlib review process へ提出する。
5. upstream merge 後は、certificate の dependency を canonical mathlib theorem へ migration できるようにする。

9.3 Academic citation

Citation は platform homepage ではなく immutable certificate version を指す。推奨 citation bundle は title、authors / attribution、certificate id、certificate hash、formal statement hash、environment epoch、access date、permanent URL、related paper DOI を含む。public attribution は privacy request により mutable であり得るため、certificate hash から個人情報を分離する。

9.4 Preservation tiers

- **Online:** site、API、certificate JSON、source artifact。
- **Exportable:** self-contained tar / OCI bundle、manifest、verification script。
- **Replicated:** operator と異なる provider / Git repository に batch root と public artifacts を mirror。
- **Migrated:** historical epoch と active epoch の両方で status を追跡。

Reproducible Builds project が示すように、同一 source だけでなく、tool、environment、instructions を記録し、第三者が再実行して artifact を比較できる経路が必要である [19]。

10 Governance and sustainable operation

10.1 Small-team operating model

TheoremForces は public documentation 上、single independent maintainer の hobby project であり SLA を持たない [2]。この制約を隠さず、同時進行 epic を最大 2、weekly triage を 30 分、monthly roadmap review を 1 回とする。

Area	Primary responsibility	Required second check
Research vision / flagship curation	mathematics lead	infrastructure feasibility
Judge / sandbox / observability	infrastructure lead	trust-policy review
Certificate schema / public claims	joint	two-person approval
Security incident	infrastructure lead	timeline and disclosure review
Academic outreach / case study	mathematics lead	participant consent
Paid pilot / legal entity	joint	gate review and external advice

10.2 Change governance

Trust-critical change は Issue、design note、tests、review、rollback plan、changelog を必要とする。特に次は two-person approval とする。

- acceptance predicate、axiom policy、wrapper template
- certificate canonicalization / hash algorithm
- environment epoch switch
- judge image、sandbox policy、secret scope
- certificate invalidation / retraction
- pricing と reputation policy

10.3 Business boundary

Public judge、public certificate、public ledger は free のまま維持する。paid feature は private ledger、collaboration、export、migration、citation tooling といった hosted software に限定し、Accepted、Seeds、ranking、review outcome、judge priority を販売しない [8]。

Paid pilot は信頼性、継続利用、support burden、data export、cost accounting の gate を満たした場合のみ開始する。初期は Personal と Team の二 tier に絞り、需要が確認される前に Pro / Pro+ / Enterprise を細分化しない。法人化は象徴的 milestone ではなく、継続売上、組織契約、liability separation、contributor payment の必要が生じた時点で判断する。

11 Roadmap: 2026-08 to 2027-07

Period	Theme	Deliverables	Exit gate
2026-08	Trustworthy beta	credential rotation、全 certificate audit、exact artifact storage、schema v2、sandbox fail-closed、stats / docs sync、backup restore drill	14 days with no critical integrity incident
2026-09 to 10	Research lighthouse	flagship collection 2、design partner 5-8、first-value onboarding、v4.32.2 shadow migration、verifier alpha	external prover 5、external dependency reuse 1
2026-11 to 2027-01	Review and retention	review dimensions、dispute / retraction、curated collections、verifier stable、monthly research session	external prover 10、reviewed 5、curated 2
2027-02 to 04	Paid pilot decision	private ledger design partner、cost model、export / deletion / downgrade、support measurement、two-tier offer	2-3 paid pilots or explicit no-launch decision
2027-05 to 07	Evidence-driven scale	redundant judge、quarterly migration、independent verifier、transparency report、research collections 3-5	external prover 25、reviewed 20、curated 10

11.1 Phase 0: first 48 hours

1. suspected exposed credentials を失効・再発行し、operator MFA を有効化。
2. misleading *latest* / *fully reproducible* label を修正。
3. certificate minting に integrity kill switch を追加。
4. public sample mismatch を incident / integrity Issue として記録。

11.2 Phase 1: trustworthy beta

最初の一か月は feature freeze とする。全 387 certificate を audit し、schema v2 と verifier を先に作る。toolchain migration は historical record を破壊せず、新 epoch で shadow build してから default を切り替える。public repository には SECURITY.md、architecture、reproduction instructions、Issue templates、Milestones を追加する。

11.3 Phase 2: research lighthouse

5-8 人の invite-only design partner と一緒に onboarding し、sign-up から first accepted certificate までは観察する。median time-to-first-certificate 30 分以内を目標とし、上位 5 blocker を修正する。research case study は「短時間で解けた」という marketing claim だけでなく、problem decomposition、human / AI contribution、failed attempts、dependency reuse、paper connection を記録する。

11.4 Phase 3: review and retention

Review quorum を実際に成立させ、外部 user が 4 週間後も戻る理由を作る。leaderboard より、collection progress、dependency unblock、migration health、review request を notification の中心にする。monthly research session では、未解決 problem 一つと certificate chain 一つに集中する。

11.5 Phase 4: paid pilot decision

Paid launch は deadline ではなく decision gate である。private ledger を 4 週間以上継続利用する design partner が 3 組以上、support が週 3 時間以内、cost / judge run と cost / active ledger が説明可能、90 日間 trust gate 維持、export / deletion / incident notification が動作する場合にだけ pilot を開く。

11.6 Phase 5: evidence-driven scale

Scale は user count ではなく verified demand に合わせる。judge redundancy、queue failover、quarterly migration、annual transparency report、independent verifier implementation を導入する。community との関係は mathlib の代替ではなく、reproducible staging / citation layer として築く。

12 Metrics and decision gates

12.1 North Star

$$\text{North Star} = \# \left\{ \begin{array}{l} \text{provenance-complete certificates authored by external users} \\ \text{and reused by another external proof or cited research output} \end{array} \right\}. \quad (5)$$

この metric は supply、external adoption、reuse、research relevance、provenance completeness を同時に要求する。

12.2 Dashboard

KPI	2026-08 gate	2027-01	2027-07
New certificate artifact hash match	100%	100%	100%
Dependency record match	100%	100%	100%
Sandbox limit enforcement	100%	100%	100%
Backup restore drill	pass	quarterly pass	quarterly pass
Distinct external accepted provers	baseline 2	10	25
Monthly retained external provers	instrument	4	10
Reviewed certificates	baseline 0	5	20
Curated certificates	baseline 0	2	10
Externally reused certificates	instrument	3	10
Public research case studies	0	2	3-5
Paper / preprint citations	instrument	1	3

Operational dashboard には p50 / p95 queue wait、judge runtime、timeout rate、infrastructure failure rate、proof failure rate、cost / run、active private ledger storage cost、weekly support time、security / integrity incident count を含める。

12.3 Feature decision rule

新機能は次の順で判定する。

1. certificate trust を上げるか。
2. real research の reuse / citation を増やすか。
3. external user の first value / retention を改善するか。
4. small part-time team で継続運営できるか。
5. measurable hypothesis があるか。

一つも満たさない場合は作らない。

13 Open questions

1. **Independent trust root:** batch root をどこへ publish すれば operator-independent verification が実用的か。
2. **Certificate semantics:** imported certificate constant を axiom として扱う実装と、proof term /

source reuse の trade-off をどう示すか。

3. **Environment policy:** monthly Lean release cadence に対し、何 release 以内を target にするか。
4. **Review quorum:** 5-person quorum を維持しつつ、初期 reviewer shortage をどう解消するか。
5. **Privacy:** private ledger の artifact preservation と user-requested deletion をどう両立するか。
6. **Persistent identifiers:** project-specific certificate id に DOI / SWHID 等を接続するか。
7. **AI benchmarking:** generated proof の model / prompt provenance を privacy と reproducibility の範囲でどこまで記録するか。
8. **Upstream bridge:** mathlib merge 後の certificate dependency migration を自動化するか。

14 Conclusion

Formal mathematics が人間だけでなく AI system によって大量に生成されるとき、価値の中心は「proof がある」という claim から、何が検査され、どの artifact と environment が使われ、どの assumptions と dependencies を持ち、誰が意味を review し、誰が再現したかという provenance graph へ移る。

TheoremForces は、その graph を公開 certificate layer として構築する機会を持つ。すでに judge、ledger、axiom report、private workspace、semantic review model の基礎がある。一方で、external adoption と review は初期であり、public artifact と provenance claim の一致には trust-critical な beta debt が残る。

したがって次の一年の勝ち筋は明確である。まず exactness を修復し、次に real research で価値を示し、その後に少人数の external community と review network を育てる。課金と scale は、その証拠の後にのみ進める。この順序を守ることで、TheoremForces は proving game ではなく、human-AI formal mathematics のための長期的な public verification infrastructure になり得る。

付録 A Proposed certificate v2 example

Listing 3: Illustrative JSON. Field names are non-normative.

```
{
  "schema": "tcl-certificate-v2",
  "certificateId": "tcl-cert-...",
  "statement": {
    "sha256": "...",
    "canonicalTextMediaType": "text/x-lean"
  },
  "submission": {
    "proofBodySha256": "...",
    "generatedSource": {
      "sha256": "...",
      "uri": "https://.../blobs/sha256/...",
      "modulePath": "Theoremforces/.../Submission_...lean",
      "templateId": "tf-wrapper-v2"
    }
  },
  "environment": {
    "epoch": "tf-lean-4.32.2-001",
    "leanToolchain": "leanprover/lean4:v4.32.2",
    "mathlibCommit": "<full commit sha>",
    "lakeManifestSha256": "..."
  },
  "builder": {
    "imageDigest": "sha256:...",
    "architecture": "linux/arm64",
```

```

    "sandboxPolicy": "tf-sandbox-v2",
    "runId": "...",
    "limitsEnforced": true
  },
  "axioms": {
    "normalized": ["Classical.choice", "Quot.sound", "propeft"],
    "rawReportSha256": "...",
    "parserVersion": "tf-axiom-parser-v2"
  },
  "dependencies": {
    "direct": [
      {
        "certificateHash": "sha256:...",
        "moduleSourceSha256": "...",
        "canonicalConstant": "Theoremforces.Ledger.Cert_....certified"
      }
    ],
    "transitiveClosureSha256": "..."
  },
  "result": {
    "conclusion": "success",
    "judgeBundleSha256": "...",
    "judgeLogSha256": "...",
    "acceptedAt": "2026-...Z"
  },
  "transparency": {
    "batchId": "...",
    "leafIndex": 42,
    "inclusionProof": ["sha256:..."],
    "signedTreeHead": "..."
  },
  "certificateHash": "sha256:..."
}

```

付録 B Verification algorithm

Listing 4: Reference verification pseudocode

```

function verify(certificateId, mode):
  c = downloadCertificate(certificateId)
  require sha256(canonicalize(withoutHash(c))) == c.certificateHash

  statement = downloadBlob(c.statement.sha256)
  proofBody = downloadBlob(c.submission.proofBodySha256)
  source = downloadBlob(c.submission.generatedSource.sha256)
  require hashesMatch(statement, proofBody, source)

  direct = resolveDependencies(c.dependencies.direct)
  require dependencyManifestHash(direct) == c.dependencies.manifestSha256
  require transitiveClosureHash(direct) == c.dependencies.transitiveClosureSha256

  require verifyMerkleInclusion(c.certificateHash, c.transparency)
  require verifySignedTreeHead(c.transparency.signedTreeHead)

  if mode in {REBUILD, MIGRATION}:
    env = materializeEnvironment(c.environment, mode)
    result = sandboxedLakeBuild(env, source, direct)
    require result.limitsEnforced
    require result.conclusion == SUCCESS
    require normalizeAxioms(result.rawAxiomReport) == c.axioms.normalized

  return VERIFIED

```

付録 C Operational SLO proposal

Public beta は formal SLA を約束しない。ただし内部 SLO を持ち、capacity と paid readiness の判断に使う。

Signal	Initial objective	Response
Certificate integrity	100%	mismatch 1 件で mint pause と audit
Sandbox preflight	100%	failure worker を pool から除外
Queue availability	99.0% monthly	status update、capacity review
p95 queue wait	under 5 min	concurrency / fairness tuning
p95 judge runtime	under 10 min	timeout taxonomy と challenge-specific review
Infrastructure error rate	under 2%	automatic retry once、root-cause bucket
Backup RPO	24 h	daily verified backup
Restore RTO	4 h	quarterly restore drill
Critical incident acknowledgement	24 h	operator notification and public status note

付録 D Glossary

- Accepted
- Recorded environment で acceptance predicate を満たした submission。数学的 endorsement ではない。
- Artifact
- Judge が実際に処理した generated Lean source と bundle contents。
- Certificate
- Artifact、environment、axiom、dependency、result を content-addressed metadata として固定した record。
- Conjecture
- User が証明対象として提示する formal target と human-readable intent。
- Environment epoch
- Lean、Mathlib、manifest、judge image、policy をまとめた versioned execution environment。
- Intent review
- Formal statement と informal mathematical claim の alignment review。
- Migration
- Historical certificate を別 environment epoch で再検査し compatibility status を記録すること。
- Provenance complete
- Exact artifact と生成・実行・依存 metadata が第三者検算可能な状態。
- Reverified
- Original acceptance とは別に、指定 environment で再度 build に成功した状態。

Seeds

Platform 内 reputation point。 money、 token、 redeemable asset ではない。

References

参考文献

- [1] TheoremForces Project, “The public certificate layer for reproducible Lean proofs,” <https://theoremforces.org/>, accessed 2026-08-07.
- [2] TheoremForces Project, “About TheoremForces,” <https://theoremforces.org/docs/about>, accessed 2026-08-07.
- [3] TheoremForces Project, “How TheoremForces judges a proof,” <https://theoremforces.org/docs/judging>, accessed 2026-08-07.
- [4] TheoremForces Project, “Security model,” <https://theoremforces.org/docs/security>, accessed 2026-08-07.
- [5] TheoremForces Project, “Trust metrics - public ledger reliability,” <https://theoremforces.org/docs/metrics>, accessed 2026-08-07.
- [6] TheoremForces Project, “Theorem Certificate Ledger,” <https://theoremforces.org/docs/ledger>, accessed 2026-08-07.
- [7] TheoremForces Project, “Semantic provenance,” <https://theoremforces.org/docs/semantic-provenance>, accessed 2026-08-07.
- [8] TheoremForces Project, “Pricing,” <https://theoremforces.org/pricing>, accessed 2026-08-07.
- [9] TheoremForces Project, “Public stats API,” <https://theoremforces.org/api/public/stats>, snapshot retrieved 2026-08-07.
- [10] TheoremForces Project, “theoremforces-judge,” GitHub repository, <https://github.com/ryendo/theoremforces-judge>, accessed 2026-08-07.
- [11] TheoremForces Project, “Submission cmq1kfbwt000xs41rafzdl4iz,” <https://theoremforces.org/submissions/cmq1kfbwt000xs41rafzdl4iz>, accessed 2026-08-07.
- [12] TheoremForces Project, “Certificate tcl-cert-20260605-0d2nux,” <https://theoremforces.org/ledger/certificates/tcl-cert-20260605-0d2nux>, accessed 2026-08-07.
- [13] Lean Project, “The Lean Language Reference,” <https://lean-lang.org/doc/reference/latest/>, accessed 2026-08-07.
- [14] Lean Project, “Elaboration and Compilation: The Kernel,” <https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/>, accessed 2026-08-07.
- [15] Lean Project, “Axioms,” <https://lean-lang.org/doc/reference/latest/Axioms/>, accessed 2026-08-07.
- [16] Lean Project, “Validating a Lean Proof,” <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>, accessed 2026-08-07.
- [17] Lean Project, “Lean 4 releases,” <https://github.com/leanprover/lean4/releases>, accessed 2026-08-07.

- [18] Lean Community, “mathlib4: The math library of Lean 4,” <https://github.com/leanprover-community/mathlib4>, accessed 2026-08-07.
- [19] Reproducible Builds Project, “Reproducible Builds,” <https://reproducible-builds.org/>, accessed 2026-08-07.
- [20] OpenSSF, “SLSA Specification v1.2: Provenance,” <https://slsa.dev/spec/v1.2/provenance>, accessed 2026-08-07.
- [21] in-toto Project, “in-toto Attestation Framework,” <https://github.com/in-toto/attestation>, accessed 2026-08-07.
- [22] B. Laurie et al., “Certificate Transparency Version 2.0,” RFC 9162, IETF, 2021, <https://www.rfc-editor.org/rfc/rfc9162.html>.